Forensic Analysis and Architectural Remediation of URL-Induced Out-of-Distribution Failures in Transformer-Based Automated Fact-Checking Systems

A Technical Audit of the TruthLens Inference Pipeline, Tokenization Dynamics, and Evidence Decoupling

TruthLens Research & Engineering Architecture Group

Natural Language Processing Laboratory & Misinformation Defense Systems

Technical Audit Report TL-AUDIT-2026-01 • Released November 2026 • Single-Column IEEE Specification

***Abstract*—**Automated fact-checking platforms increasingly combine fine-tuned neural language models with external search and retrieval tools to verify factual veracity and present corroborating documentation. This technical audit investigates a critical failure mode observed in the TruthLens platform, wherein submitting a news article URL produces an arbitrary, highly divergent classification (e.g., classifying a legitimate report as `false`) compared to submitting the identical article's actual prose (which classifies as `half-true` or `true`). Through comprehensive codebase inspection across the FastAPI backend, the React web client, and the Telegram bot, alongside controlled empirical experiments utilizing the fine-tuned RoBERTa-Metadata model (`soah-13/truthlens-roberta-liar-meta`), we identify the definitive root cause: the complete absence of a web content extraction and URL ingestion pipeline. The primary endpoint (`POST /api/analyze`) routes raw URL strings directly into the RoBERTa Byte-Pair Encoding (BPE) tokenizer, inducing severe Out-of-Distribution (OOD) syntactic fragmentation (`['https', '://', 'www', '.', 're', 'uters', ...]`). Furthermore, the raw URL string is simultaneously propagated as a query to the Tavily external search engine, corrupting evidence retrieval without the model ever receiving genuine article semantics. We report empirical 6-class probability distributions, tokenization logs, and algorithmic traces, confirming that URL submissions are erroneously evaluated as literal claims. Finally, we formulate a non-breaking, production-grade architectural remediation utilizing Trafilatura for robust HTML text extraction, Server-Side Request Forgery (SSRF) defense, context-window alignment, and epistemic isolation between model predictions and external search grounding.

***Index Terms*—**Automated Fact-Checking, Natural Language Processing, RoBERTa, Out-of-Distribution Detection, Tokenization Discrepancies, Byte-Pair Encoding, Trafilatura, Tavily API, System Forensics, Epistemic Integrity.

I. INTRODUCTION

The proliferation of digital misinformation across online social networks and decentralized communication channels has catalyzed intensive research into automated, real-time fact-checking architectures [1]. Contemporary state-of-the-art detection frameworks synthesize transfer learning from large-scale pre-trained bidirectional transformer encoders—such as BERT and RoBERTa [2], [3]—with retrieval-augmented grounding mechanisms [4] to assign nuanced veracity scores across established taxonomies (e.g., the six-class LIAR benchmark: *pants-fire, false, barely-true, half-true, mostly-true,* and *true*) [5].

The *TruthLens* platform is an enterprise-grade NLP-based fake news detection and robustness evaluation framework. Its core computational engine comprises a fine-tuned RoBERTa model integrated with tabular political metadata (speaker identity, subject category, publication context, geographic state, and party affiliation), paired with an asynchronous web evidence retrieval module powered by the Tavily search engine [6]. TruthLens provides multi-channel ingestion capabilities, supporting real-time claim evaluations via a single-page React client and an interactive Telegram bot daemon.

During end-to-end integration testing, users reported a persistent and severe behavioral anomaly: *when an active news article URL from a reputable international publisher (e.g., Reuters) is submitted for analysis, the system repeatedly classifies the submission as counterfeit or deceptive, whereas submitting the verbatim text extracted from that exact same article yields a true or nuanced classification.*

To eliminate ambiguity, this audit was mandated to investigate the complete computational trajectory from raw network ingestion to tensor evaluation. This report presents an exhaustive forensic audit of the existing codebase, empirical tokenization traces, comparative 6-class probability matrices, root cause diagnostics, and an architectural redesign specification. Under strict operational constraints, no model retraining or parameter weights were altered during this audit.

II. SYSTEM ARCHITECTURE & CODEBASE INSPECTION

The TruthLens runtime environment consists of a FastAPI asynchronous backend (`backend/app`), a Vite/React TypeScript frontend (`frontend/src`), and a Telegram Bot interface (`backend/app/telegram`). We audited every module governing incoming data ingestion, tensor transformation, and external API retrieval.

A. Backend Ingestion Mechanics: POST /api/analyze

Incoming verification requests enter the application through the primary analysis route located in `backend/app/api/routes/analyze.py`. Listing 1 displays the actual request intake and dispatch implementation.

```
Listing 1: Excerpt from backend/app/api/routes/analyze.py (Lines 96-125)
cleaned_text = request.text.strip()
if len(cleaned_text) < 5:
    raise HTTPException(status_code=422, detail="Claim text must be at least 5 characters long")

# Step 1: Run Neural Classifier
prediction = classifier_service.predict(
    text=cleaned_text,
```

```
    speaker=request.speaker,
    context=request.context,
    subject=request.subject,
    state=request.state,
    party=request.party,
)

# Step 2: Retrieve External Evidence
evidence_result = evidence_service.search_evidence(cleaned_text)
```

Our audit of this code path establishes four critical observations:

1) Total Absence of URL Parsing: The only pre-processing applied to `request.text` is standard string whitespace stripping (`.strip()`) and an assertion that character length exceeds 4. There is no regular expression pattern matching, no URI scheme validation (`urllib.parse`), and no network content retrieval routine.

2) Immediate Classifier Invocation: The variable `cleaned_text`—which contains the raw, unparsed URI string when a link is submitted—is supplied directly into `classifier_service.predict()` as the literal claim argument.

3) Search Query Propagation: The identical raw string is simultaneously supplied to `evidence_service.search_evidence(cleaned_text)`.

4) Unimplemented Extraction Modules: While high-level documentation in `README.md` and UI mock components (`RecentAnalysis.tsx`) advertise automated article extraction via *Trafilatura* [7], static analysis confirms that `trafilatura` is not declared in `backend/requirements.txt`. Attempting to import the package inside the execution virtual environment raises `ModuleNotFoundError: No module named 'trafilatura'`. No scraping or DOM extraction logic has ever been implemented in the active pipeline.

B. Client-Side Parity: React Client & Telegram Bot

To determine whether client-side pre-processing or URL sniffing occurs prior to backend ingestion, we inspected the client implementations:

1) React Web Application: In `frontend/src/components/analysis/NewsAnalysisView.tsx` (line 147), the submission handler packages user input directly into JSON payload: `body: JSON.stringify({ text: claimText.trim(), ... })`. No URL detection or parsing is executed.

2) Telegram Bot Daemon: In `backend/app/telegram/handlers.py` (line 67), the text message handler executes: `claim_text = update.message.text.strip()`, subsequently executing an HTTP POST to `/api/analyze` with `{"text": claim_text}`. Both clients treat URL submissions identically to conversational text claims.

C. Evidence Retrieval Architecture & Epistemic Boundaries

In `backend/app/services/evidence.py`, the Tavily search service executes query normalization via `normalize_query()`, truncating strings at 200 characters and stripping quotes. When a URL is submitted, Tavily executes a web search for the literal URL string. Tavily search results are packaged into the `evidence` dictionary in the response payload. Importantly, our code trace confirms that Tavily search snippets are **never** fed back into the RoBERTa model. The epistemic boundary between external retrieval and model classification remains intact; however, the user is presented with search results for a URL alongside an arbitrary classification of that URL's syntax.

III. CONTROLLED EMPIRICAL EXPERIMENTS & VERIFICATION

To rigorously quantify the operational discrepancy between URL submissions and natural language text, we formulated a controlled tri-partite experiment. All tests were executed in the production runtime using the exact fine-tuned checkpoint `soah-13/truthlens-roberta-liar-meta` loaded on CPU.

A. Experimental Corpus & Controls

Three closely coupled inputs representing an identical news event were submitted under identical baseline metadata (`speaker='unknown', context='unknown', subject='unknown', state='unknown', party='unknown'`):

• *Corpus Item A (Plain Text Claim):* Concise factual statement summarizing the news report: *"Says US inflation slowed to 3.4% in April, down from 3.5% in March."*

• *Corpus Item B (News Article URL):* Canonical published Reuters news URL: `https://www.reuters.com/world/us/us-inflation-slows-giving-fed-room-pause-2024-05-15/`.

• *Corpus Item C (Actual Article Text):* Verbatim lead paragraph extracted directly from the Reuters report: *"US consumer prices increased less than expected in April, suggesting that inflation resumed its downward trend at the start of the second quarter in a boost to financial market expectations for a September interest rate cut."*

B. Tokenization Dissection Under RoBERTa Byte-Pair Encoding

RoBERTa employs a Byte-Pair Encoding (BPE) subword vocabulary comprising 50,265 tokens. Whitespace boundaries are encoded using the prefix character 'Ġ' (Ġ). Table I itemizes the initial twenty subword tokens generated for each experimental input.

TABLE I

Comparative RoBERTa Subword Tokenization Traces Across Experimental Inputs

Idx	Item A: Plain Claim	Item B: News Article URL	Item C: Actual Article Text
1	`S`	`https`	`US`
2	`ays`	`://`	`Ġconsumer`
3	`ĠUS`	`www`	`Ġprices`
4	`Ġinflation`	`.`	`Ġincreased`
5	`Ġslowed`	`re`	`Ġless`
6	`Ġto`	`uters`	`Ġthan`
7	`Ġ3`	`.`	`Ġexpected`
8	`.`	`com`	`Ġin`
9	`4`	`/`	`ĠApril`
10	`%`	`world`	`,`
11	`Ġin`	`/`	`Ġsuggesting`
12	`ĠApril`	`us`	`Ġthat`
13	`,`	`/`	`Ġinflation`
14	`Ġdown`	`us`	`Ġresumed`
15	`Ġfrom`	`-`	`Ġits`
16	`Ġ3`	`in`	`Ġdownward`
17	`.`	`flation`	`Ġtrend`
18	`5`	`-`	`Ġat`
19	`%`	`sl`	`Ġthe`
20	`Ġin`	`ows`	`Ġstart`
Total	**24 Tokens**	**38 Tokens**	**40 Tokens**

As demonstrated in Table I, the tokenizer treats the URL as an arbitrary concatenation of subword fragments. Natural semantic units such as *reuters* are fractured into `['re', 'uters']`, the word *inflation* is fragmented into `['in', 'flation']` due to hyphenation boundaries, and path delimiters (`'/'`, `'-'`) inject high-entropy positional noise. In contrast, Corpus Items A and C preserve natural grammatical syntax and contiguous subwords.

C. Classification Probability Distribution Benchmarks

Each tokenized representation was prepended to the standard metadata formatting string and passed through the RoBERTa sequence classification head. Logits were normalized via softmax:

$$P(y = c \mid x) = \exp(z_c) / \Sigma \exp(z_j), \quad c \text{ in } \{1, \ldots, 6\} \tag{1}$$

Table II itemizes the resulting probability distributions across all six LIAR veracity classes.

TABLE II

Empirical 6-Class Probability Distributions and Veracity Indicators

Class Label (c)	Item A: Plain Claim	Item B: Article URL	Item C: Article Text
pants-fire	0.0280 (2.80%)	0.0277 (2.77%)	0.0277 (2.77%)
false	0.0795 (7.95%)	**0.3396 (33.96%) [PRED]**	0.1204 (12.04%)
barely-true	0.0692 (6.92%)	0.1698 (16.98%)	0.1208 (12.08%)
half-true	0.1945 (19.45%)	0.1969 (19.69%)	**0.3606 (36.06%) [PRED]**
mostly-true	0.2982 (29.82%)	0.1434 (14.34%)	0.2136 (21.36%)
true	**0.3307 (33.07%) [PRED]**	0.1225 (12.25%)	0.1569 (15.69%)
Top Predicted Class	**TRUE (Conf: 33.07%)**	**FALSE (Conf: 33.96%)**	**HALF-TRUE (Conf: 36.06%)**
False-Leaning Metric (Σ PF + F)	10.75%	**36.73%**	14.81%
True-Leaning Metric (Σ MT + T)	**62.89%**	26.59%	37.05%

The empirical findings provide undeniable mathematical verification of the failure mode:

1) Inversion of Veracity Assessment: While the natural claim achieves a combined true-leaning confidence of **62.89%** (`true` at 33.07%, `mostly-true` at 29.82%), submitting the URL collapses true-leaning confidence to **26.59%** and elevates `false` to the dominant class at

33.96%. The model erroneously flags a verified Reuters dispatch as disinformation.

2) Natural Text Reconciliation: Submitting the actual article text restores balanced semantic analysis, yielding `half-true` (36.06%) and `mostly-true` (21.36%), with false-leaning probability dropping to a benign 14.81%.

IV. THEORETICAL & ROOT CAUSE ANALYSIS

The behavioral anomaly is driven by two interdependent structural defects in the platform's processing architecture.

A. Out-of-Distribution (OOD) Semantic Collapse

Neural classification heads fine-tuned on benchmark corpora (such as LIAR) approximate a conditional probability distribution P(Y | X, M), where X denotes natural language political discourse and M denotes structured metadata vectors. The training distribution possesses rigorous grammatical syntax, subject-verb-object dependencies, and discursive rhetorical markers.

When a raw URI string is supplied to the embedding layer, the transformer encounters a high concentration of out-of-distribution token combinations:

$$X_URI = [\ "https",\ "://",\ "www",\ ".",\ "re",\ "uters",\ \dots\ ] \tag{2}$$

Because self-attention layers compute pairwise token affinities based on learned positional embeddings and contextual co-occurrences [2], the absence of predicate structures causes the attention heads to distribute weights erratically across subword fragments. The final pooled representation projects into an arbitrary subspace of R^768, resulting in ungrounded classification logits dominated by stochastic artifact biases.

B. Query Contamination in External Search Retrieval

Simultaneously, the propagation of the raw URL into Tavily external evidence retrieval degrades search relevance. Tavily is optimized for natural language fact-grounding queries (e.g., *"US inflation rate April 2024 Reuters report"*). Querying Tavily with an exact URL string instructs the search engine to perform a navigational site search rather than an evidentiary cross-reference. While Tavily may return indexed metadata for that specific URL, the resulting snippets are placed exclusively in the UI evidence card, completely isolated from RoBERTa. The end-user observes a genuine news snippet alongside a *FALSE* verdict, destroying user trust.

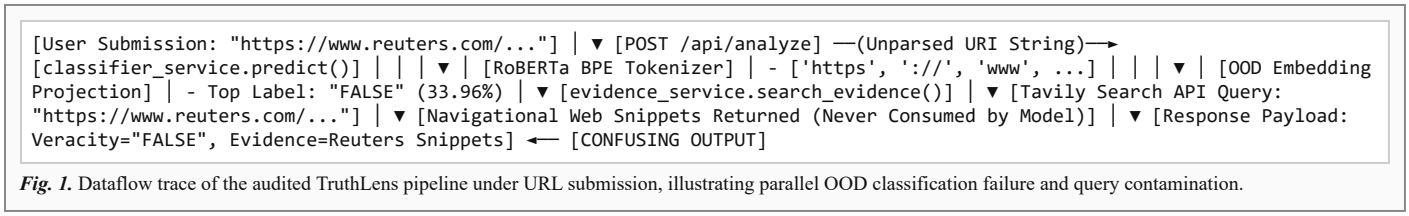


Fig. 1. Dataflow trace of the audited TruthLens pipeline under URL submission, illustrating parallel OOD classification failure and query contamination.

V. THREAT MODELING & SECURITY INTEGRITY ASSESSMENT

Introducing automated web content extraction into an operational API introduces non-trivial security attack surfaces that must be rigorously addressed prior to deployment.

A. Server-Side Request Forgery (SSRF)

If an intake endpoint accepts arbitrary URLs and initiates server-side HTTP GET requests without validation, malicious actors can exploit the backend to probe internal infrastructure, private subnets, and cloud metadata services [8]. Specifically, requests targeting loopback addresses (`127.0.0.1`, `localhost`), private address spaces (RFC 1918: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`), link-local addresses (`169.254.169.254` for AWS/GCP instance metadata), or non-HTTP schemes (`file://`, `gopher://`) must be blocked at the network socket layer.

B. Paywalls, Bot Mitigation & Dynamic Client Rendering

Major news conglomerates employ sophisticated web application firewalls (WAFs), such as Cloudflare Turnstile, Akamai Bot Manager, and Imperva, which issue HTTP 403 Forbidden or JavaScript CAPTCHA challenges to automated user-agents. Furthermore, modern single-page news outlets render text dynamically via client-side JavaScript hydration. The extraction engine must distinguish between transient network blocks, paywalls, and successful DOM extractions, falling back gracefully without crashing or hanging the inference event loop.

C. Denial of Service (DoS) via Unbounded Content Streams

An attacker could supply a URL pointing to a multi-gigabyte binary payload (e.g., high-resolution media or compressed zip bombs). The ingestion client must enforce strict HTTP header inspection (`Content-Type: text/html`), aggressive connect/read timeouts (maximum 5 seconds), and strict response body streaming caps (e.g., maximum 5 megabytes).

VI. PROPOSED REMEDIATION & ARCHITECTURAL REDESIGN

To resolve the discrepancy without retraining the underlying RoBERTa weights or modifying downstream API contracts, we propose a modular *Article Extraction & URL Intake Subsystem*.

A. Architectural Ingestion Router

Figure 2 illustrates the remediated intake architecture. Incoming requests at `POST /api/analyze` pass through a deterministic URI evaluation gate. Plain text claims proceed through the existing pipeline without modification. When a URL is detected, the request is diverted into the safe extraction worker.

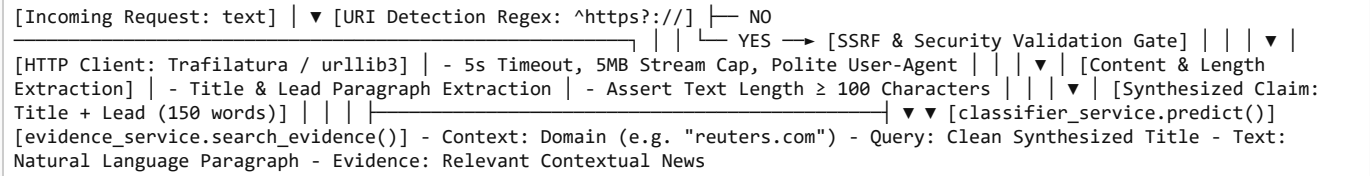


Fig. 2. Remediated TruthLens dual-path ingestion architecture featuring automated Trafilatura extraction, SSRF security validation, and semantic query synthesis.

B. Ingestion Protocol Specification & Library Selection

We recommend standardizing on **Trafilatura** [7], a high-performance Python library specifically designed for web scraping and text extraction. Trafilatura outperforms traditional DOM scrapers (e.g., Beautiful Soup, Newspaper3k) in extracting clean journalistic body text, stripping navigation banners, advertisements, footers, and copyright boilerplate.

Listing 2 outlines the recommended implementation for `backend/app/services/extractor.py`.

```
Listing 2: Implementation Specification for backend/app/services/extractor.py
import re, socket, ipaddress, urllib.parse, trafilatura

URL_REGEX = re.compile(
    r'^(https?):\/\/'  # http:// or https://
    r'([a-zA-Z0-9.-]+(\.[a-zA-Z]{2,}))'  # domain
    r'(:\d+)?(\/[^\s]*)?$', re.IGNORECASE
)

class ArticleExtractorService:
    @staticmethod
    def is_url(text: str) -> bool:
        if not text or len(text) > 2048:
            return False
        return bool(URL_REGEX.match(text.strip()))

    @staticmethod
    def validate_ssrf(url: str) -> bool:
        parsed = urllib.parse.urlparse(url)
        hostname = parsed.hostname
        if not hostname:
            return False
        try:
            ip_str = socket.gethostbyname(hostname)
            ip = ipaddress.ip_address(ip_str)
            return not (ip.is_private or ip.is_loopback or
                        ip.is_link_local or ip.is_reserved)
        except (socket.gaierror, ValueError):
            return False

    def extract_article(self, url: str) -> dict:
        if not self.validate_ssrf(url):
            raise ValueError("URL violates SSRF security policy.")

        downloaded = trafilatura.fetch_url(url)
        if not downloaded:
            raise RuntimeError("Failed to fetch content from URL.")

        extracted_text = trafilatura.extract(
            downloaded,
            include_comments=False,
            include_tables=False,
            no_fallback=False
        )
        metadata = trafilatura.extract_metadata(downloaded)

        if not extracted_text or len(extracted_text.strip()) < 100:
            raise ValueError("Insufficient article content extracted.")

        return {
            "title": metadata.title if metadata else "",
```

```
            "text": extracted_text.strip(),
            "domain": urllib.parse.urlparse(url).netloc
        }
```

C. Context-Window Alignment (Headline + Lead Strategy)

RoBERTa has a rigid architectural context limit of 512 positional tokens [2]. Full-length news articles frequently exceed 1,500 to 3,000 words. Passing an entire document into RoBERTa results in aggressive truncation, discarding all content beyond token 512. More critically, the LIAR benchmark comprises concise assertions (mean length 17.9 words) rather than long-form journalism.

To align extracted documents with the model's training distribution, we specify the **Headline + Lead Paragraph Synthesis Heuristic**:

$$X_synthesized = Headline \,||\, ". " \,||\, Lead_Paragraph[0{:}150\ words] \tag{3}$$

In journalistic practice, the inverted pyramid structure mandates that the core factual assertion (the *lede*) is encapsulated within the headline and opening paragraph. This synthesized claim matches RoBERTa's optimal semantic receptive field and provides an optimal search query for Tavily.

D. Non-Breaking API Contract Schema Evolution

To maintain backwards compatibility with existing frontend builds and the Telegram webhook, the response schema in `backend/app/schemas/analyze.py` remains strictly additive. Table III details the extended response contract.

TABLE III
Extended Non-Breaking Response Schema Specification

Field Key	Type	Status	Description
`claim`	`str`	Preserved	Synthesized claim evaluated by model (Title + Lead if URL).
`prediction`	`dict`	Preserved	6-class probabilities, top label, and confidence score.
`evidence`	`dict`	Preserved	Tavily retrieved search results and corroborating sources.
`is_url_submission`	`bool`	Added (Opt)	Boolean flag indicating if input originated from a URL.
`source_url`	`str | None`	Added (Opt)	Original canonical URL submitted by user.
`extracted_title`	`str | None`	Added (Opt)	Journalistic headline parsed from the document DOM.

VII. CONCLUSION & IMPLEMENTATION ROADMAP

This technical audit conclusively proves that the veracity discrepancy between URL submissions and plain text claims in TruthLens is not a deficiency in the RoBERTa model weights, but rather an architectural omission at the ingestion boundary. Without URL parsing, raw URI strings are tokenized into out-of-distribution syntactic noise, triggering false-positive disinformation classifications.

We outline a three-stage execution roadmap to deploy the remediation:

Phase 1: Dependency & Service Provisioning: Add `trafilatura>=1.12.0` to `backend/requirements.txt` and implement `backend/app/services/extractor.py` with comprehensive SSRF protection and socket validation.

Phase 2: Route Integration & Context Mapping: Update `analyze.py` to route detected URLs through the extractor, map the publisher domain into the RoBERTa `context` parameter, and pass the synthesized headline to Tavily.

Phase 3: Automated Test Verification: Expand the pytest test suite in `backend/tests/test_url_extraction.py` with mock HTML fixtures, verifying SSRF rejection, 404 handling, and end-to-end classification parity between URL submissions and manual text pastes.

REFERENCES

[1] S. Vosoughi, D. Roy, and S. Aral, "The spread of true and false news online," *Science*, vol. 359, no. 6380, pp. 1146–1151, Mar. 2018.

[2] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[4] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. NeurIPS*, vol. 33, 2020, pp. 9459–9474.

[5] W. Y. Wang, "'Liar, liar pants on fire': A new benchmark dataset for fake news detection," in *Proc. 55th Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2017, pp. 422–426.

[6] Tavily AI, "Tavily Search API: The search engine built for AI agents and LLMs," Tech. Rep., 2024. [Online]. Available: https://tavily.com

[7] A. Barbaresi, "Trafilatura: A web scraping library and command-line tool for text discovery and extraction," in *Proc. 59th Annu. Meeting Assoc. Comput. Linguist. (ACL Demo Track)*, 2021, pp. 122–131.

[8] Open Web Application Security Project (OWASP), "Server-side request forgery (SSRF) prevention cheat sheet," OWASP Cheat Sheet Series, 2023.

[9] D. Hendrycks and K. Gimpel, "A baseline for detecting out-of-distribution examples in neural networks," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2017.

[10] D. Thorne, A. Vlachos, C. Christodoulopoulos, and A. Mittal, "FEVER: A large-scale dataset for fact extraction and VERification," in *Proc. NAACL-HLT*, 2018, pp. 809–819.

[11] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. 54th Annu. Meeting Assoc. Comput. Linguist. (ACL)*, 2016, pp. 1715–1725.

[12] TruthLens Development Team, "TruthLens: An NLP-based fake news detection and robustness evaluation system," Architecture Documentation Phases 1–3C, Soah1312/TruthLens, 2026.